SOLVE-IT Exercise: Using generation scripts

Overview for instructors

Version	0.1 DRAFT
Author	Chris Hargreaves
Prerequisites	● git, python3, pip ● Excel (or similar) to view output
Intended Learning Outcomes (ILOs)	At the end of this exercise you will be able to: ● Clone the SOLVE-IT repository ● Install requirements needed to run SOLVE-IT scripts ● Run scripts to convert the JSON data from the knowledge base into other formats
Instructor notes	This will get students started with the SOLVE-IT repository, cloning the repo, installing requirements, and running the scripts to generate content.

SOLVE-IT Exercise: Using generation scripts

Pre-requisites

- git, python3, pip
- Excel (or similar) to view output

Aim

The overall aim of this exercise is to get you started with the SOLVE-IT scripts so that you can download the repository, install requirements, and run the scripts that convert the repository JSON data into other formats.

Note that this is not required to view the knowledge base content as these scripts are run automatically and the output is stored within the repository for ease of viewing, but if you are going on to develop in SOLVE-IT, then doing this yourself, will be needed.

Overview of the Tasks

- Cloning the repository
- Installing requirements
- Running the Excel generation script
- Running the markdown generation script

Cloning the Repository

There are a number of ways of cloning the GitHub repository:

- Using the git command at the terminal (with URL from web browser)
- Using the desktop application
- Using the GitHub command line tools

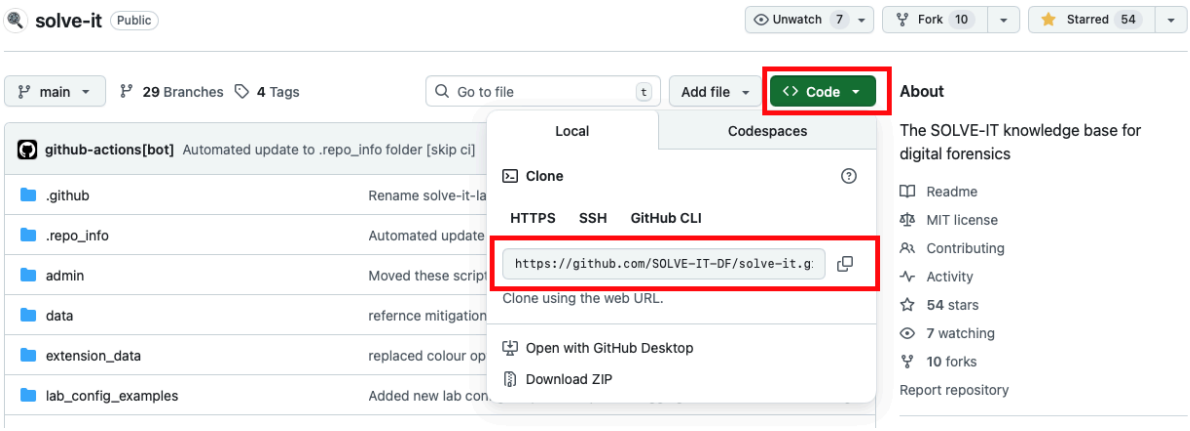
GitHub has detailed instructions for all of these methods for Mac, Windows, and Linux: https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository.

For this example we will use the git command at the terminal.

> **NOTE: Having *git* installed was in the pre-requisites. If you do not have it installed on your computer, you can find instructions here for various platforms: https://github.com/git-guides/install-git**

1. Open the terminal or command line interface on your computer

2. Go to the SOLVE-IT repository (https://github.com/SOLVE-IT-DF/solve-it). The green ‘Code’ button will have a URL to the repository, and a button that will copy the address to your clipboard.



Copy the url and go back to your terminal window. Type the command ‘git clone’ followed by pasting in the URL as illustrated below.

```
$: git clone https://github.com/SOLVE-IT-DF/solve-it.git
Cloning into 'solve-it'...
remote: Enumerating objects: 4012, done.
remote: Counting objects: 100% (1123/1123), done.
remote: Compressing objects: 100% (365/365), done.
remote: Total 4012 (delta 927), reused 790 (delta 757), pack-reused 2889 (from 1)
Receiving objects: 100% (4012/4012), 5.99 MiB | 8.54 MiB/s, done.
Resolving deltas: 100% (2729/2729), done.
$:
```

If you change directory into that new ‘solve-it’ folder and list the file contents with the `ls` or `dir` commands, you will see something similar to the content as shown below.

```
$: cd solve-it
$: ls
admin                 extension_data        README.md             solve_it_library      tests
CONTRIBUTING.md       lab_config_examples   reporting_scripts     solveitcore.py
data                  LICENSE               requirements.txt      STYLE_GUIDE.md
$:
```

Installing Requirements

At the time of writing, there are not many requirements for SOLVE-IT. They can be viewed in the downloaded content in the file `requirements.txt`. As shown below, only *pydantic* (for data validation) and *xlxswriter* (for generating Excel workbooks) are needed.

```
[$: cd solve-it
[$: ls
 admin                  extension_data         README.md              solve_it_library       tests
 CONTRIBUTING.md        lab_config_examples    reporting_scripts      solveitcore.py
 data                   LICENSE                requirements.txt       STYLE_GUIDE.md
[$: cat requirements.txt
 pydantic>=2.0.0
 xlsxwriter
 $:
```

Python requirements can be installed using the tool pip, and best practice is to do this in a 'virtual environment' (see https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/ for more details).

On Mac and Linux, this can be done using:

```
python3 -m venv .venv
```

On Windows, this can be done using:

```
py -m venv .venv
```

You can then activate your virtual environment using:

Mac/Linux:

```
source .venv/bin/activate
```

Windows:

```
.venv\Scripts\python
```

The macOS results are shown below:

```
[$: python3 -m venv .venv
[$: source .venv/bin/activate
 (.venv) $:
```

Note, to exit the virtual environment use the command `deactivate`

Now, after creating and activating your virtual environment, you can install requirements with:

Mac/Linux:

`python3 -m pip install -r requirements.txt`

Windows:
`py -m pip install -r requirements.txt`

You will see something like the text below describing the installation of the packages.

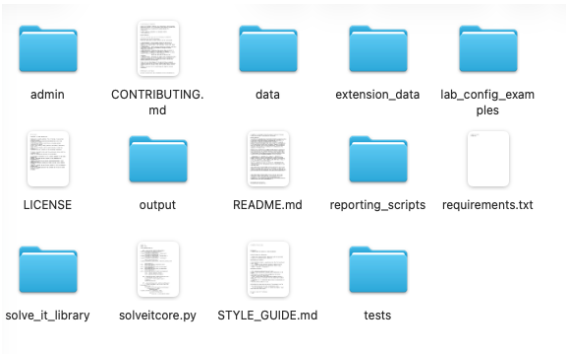
```
(.venv) $: python3 -m pip install -r requirements.txt
Collecting pydantic>=2.0.0 (from -r requirements.txt (line 1))
  Downloading pydantic-2.12.4-py3-none-any.whl.metadata (89 kB)
Collecting xlsxwriter (from -r requirements.txt (line 2))
  Using cached xlsxwriter-3.2.9-py3-none-any.whl.metadata (2.7 kB)
Collecting annotated-types>=0.6.0 (from pydantic>=2.0.0->-r requirements.txt (line 1))
  Using cached annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.41.5 (from pydantic>=2.0.0->-r requirements.txt (line 1))
  Downloading pydantic_core-2.41.5-cp313-cp313-macosx_11_0_arm64.whl.metadata (7.3 kB)
Collecting typing-extensions>=4.14.1 (from pydantic>=2.0.0->-r requirements.txt (line 1))
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting typing-inspection>=0.4.2 (from pydantic>=2.0.0->-r requirements.txt (line 1))
  Using cached typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)
Downloading pydantic-2.12.4-py3-none-any.whl (463 kB)
Downloading pydantic_core-2.41.5-cp313-cp313-macosx_11_0_arm64.whl (1.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.9/1.9 MB 8.9 MB/s eta 0:00:00
Using cached xlsxwriter-3.2.9-py3-none-any.whl (175 kB)
Using cached annotated_types-0.7.0-py3-none-any.whl (13 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Using cached typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Installing collected packages: xlsxwriter, typing-extensions, annotated-types, typing-inspection, pydantic-core, pydantic
Successfully installed annotated-types-0.7.0 pydantic-2.12.4 pydantic-core-2.41.5 typing-extensions-4.15.0 typing-inspection-0.4.2 xlsxwriter-3.2.9
```

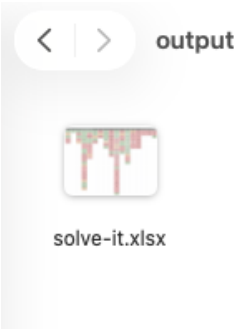
Running the Excel generation script

With these steps complete, generating the Excel workbook simply involves running the `generate_excel_from_kb.py` script that is contained within the `reporting_scripts` folder. This is shown below:

```
(.venv) $: python3 reporting_scripts/generate_excel_from_kb.py
INFO: Loaded 149 techniques.
INFO: Loaded 252 weaknesses.
INFO: Loaded 204 mitigations.
INFO: Building reverse indices for performance optimization...
INFO: Reverse indices built: 250 weakness->technique, 204 mitigation->weakness, 204 mitigation->technique
INFO: Loaded objective mapping 'solve-it.json' with 19 objectives.
Using configuration file: solve-it.json
Output will be to: output/solve-it.xlsx
Creating worksheets...
- added 'main' worksheet
- populated 'all techniques' worksheet
- populated 'all techniques (sorted)' worksheet
- populated 'all techniques' worksheet
- populated 'all weaknesses' worksheet
- populated 'all mitigations' worksheet
Updating 'main' worksheet with links to techniques...
- 'main' worksheet updated
Adding the individual techniques sheets...
- all individual techniques worksheets updated
Workbook generation complete.
(.venv) $:
```

With these options, the resulting Excel workbook will be found in a created ‘output’ folder in the root of the cloned repository. Note that an additional parameter `-o` can be passed to the script to change the output path and name of the generated workbook.

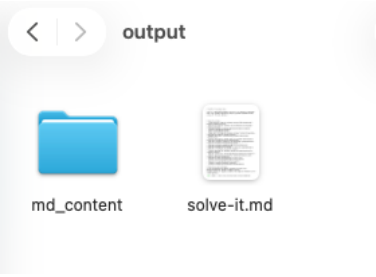




Running the markdown generation script

Similarly, the markdown content can be generated with the script reporting_scripts/generate_md_from_kb.py as shown below:

```
(.venv) $: python3 reporting_scripts/generate_md_from_kb.py
Using configuration file: solve-it.json
No extensions configured
Markdown file created at: output/solve-it.md
(.venv) $:
```



This can be viewed in any markdown viewer.